HomeHero

Stazione di Monitoraggio Ambientale Intelligente

Smart Weather Station con Dashboard IoT

Bernatskyi · Ribić · Siraj · Rossi · De Togni

26 marzo 2026

Chi siamo e cosa abbiamo costruito

IL TEAM S.A.R.R.L.

Roman Bernatskyi
Backend Lead & Architetto

Luka Ribić
Frontend Lead & Project Manager

Shaeek Siraj
Hardware & Firmware

Matteo Rossi
QA & Documentazione

Andrea De Togni
UI/UX & Analisi economica

HOMEHERO IN UNA FRASE

"Un ESP32 fisico legge **6 parametri ambientali** ogni **10 secondi**, li invia a una **REST API su cloud**, li salva in **PostgreSQL**, e li mostra in una **dashboard React** accessibile da browser."

Concetto Chiave

Ogni membro ha lavorato su una parte **separata** e tutto si integra in un **unico sistema**.

6	10s	3
Sensori	Intervallo	Cloud

Come è cresciuto il team — Modello di Tuckman

FORMING
dic 2025 – gen 2026

Team formato, ruoli assegnati, idea definita

→

STORMING
gen – feb 2026

Prime tensioni, distribuzione carichi di lavoro — normale in ogni team

→

NORMING
feb 2026

Stack condiviso, workflow Git definito

→

PERFORMING
feb – mar 2026

Massima produttività, sistema in deploy

→

ADJOURNING
mar 2026

Chiusura, presentazione

Forming

Ci siamo incontrati, abbiamo scelto il nome e i ruoli

Storming

Tensioni sulla distribuzione del lavoro — normale

Norming

Abbiamo trovato un ritmo comune — stack approvato

Performing

Tutti i componenti sviluppati e integrati in parallelo

Adjourning

Deploy completato, documentazione finita

Matrice RACI — Chi ha fatto cosa

Responsabilità per Attività (Top 12)

#	Attività	Roman	Luka	De Togni	Matteo	Shaeek
1	Idea e requisiti	R	A	C	C	C
2	Architettura	A/R	C	I	I	C
3	Backend Laravel	A/R	C	I	I	I
4	Database	A/R	C	I	I	C
5	Firmware ESP32	C	I	I	I	A/R
6	Hardware	C	I	I	I	A/R
7	Frontend React	I	A/R	C	I	I
8	UI/UX Design	C	C	A/R	I	I
9	Deploy	A/R	R	I	I	I
10	QA / Test finale	C	C	C	A/R	C
11	Documentazione	C	C	C	A/R	I
12	GANTT / RACI / Analisi	C	I	A/R	I	I

R – RESPONSIBLE
Esegue il lavoro

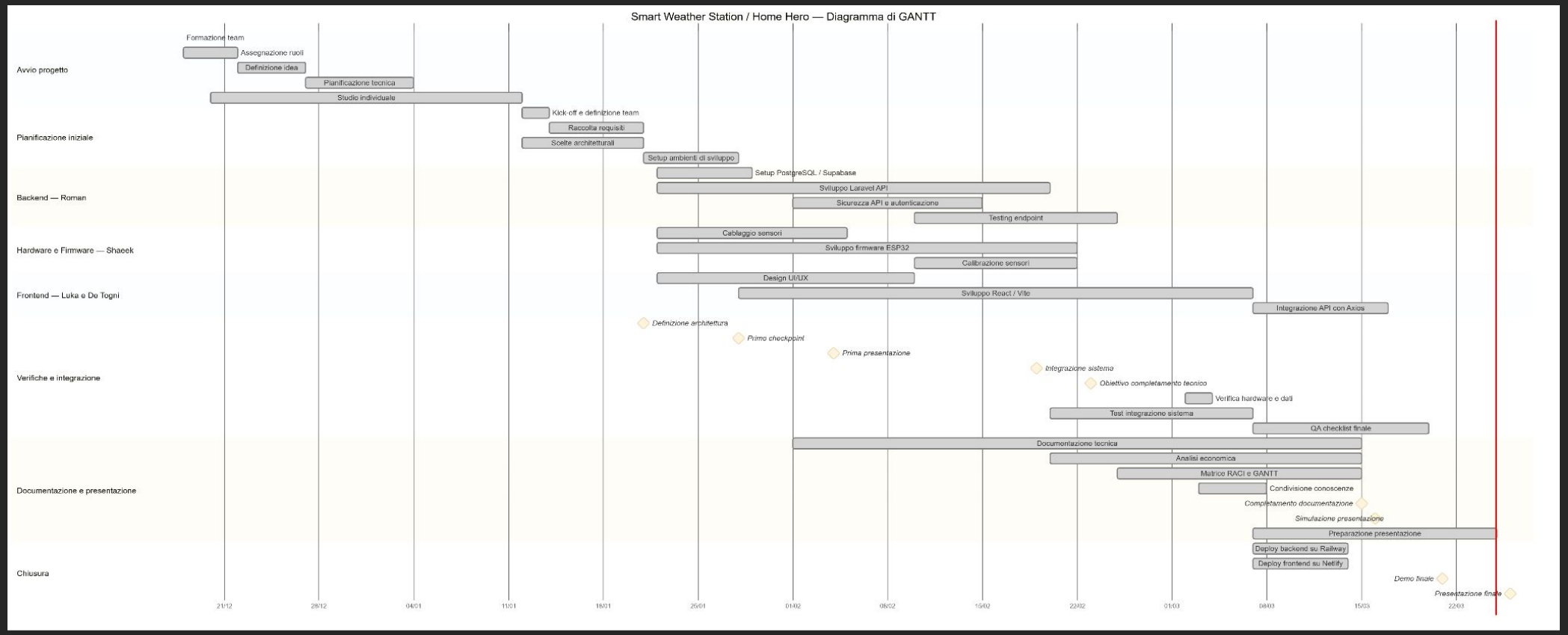
A – ACCOUNTABLE
Risponde del risultato (unico per attività)

C – CONSULTED
Fornisce input e feedback

I – INFORMED
Aggiornato sull'avanzamento

GANTT — Dalla formazione del team alla presentazione

Timeline del Progetto



Inizio Progetto
18 Dic
2025 — Formazione team

9 Milestone
Principali
Non solo "presentazione"

Deploy
14 Mar
2026 — 12 giorni prima

Stato
100%
Tutto completato secondo piano iniziale

Sviluppo parallelo : Backend + Firmware + Frontend sviluppati contemporaneamente → coordinamento critico

Architettura di sistema — Come parlano le componenti

ESP32 WROVER + 6 sensori
2 OLED · relay ventola · firmware
Shaeek

HTTPS POST ogni 10 s
(risposta con fan_desired_state)

Laravel 12 REST API
PHP 8.2 · Sanctum · Docker
Roman

Eloquent ORM

PostgreSQL su Supabase
3 tabelle: users, telemetries, device_settings
Roman

Axios (Bearer token)

React 19 + Vite 7 Dashboard
TailwindCSS 4 · Recharts · Netlify
Luka

Punti Chiave

1. L'ESP32 è il **client** HTTP, non il server — è lui che "parla per primo"
2. Il controllo ventola **non ha endpoint dedicato**: arriva come risposta JSON alla POST dell'ESP32
3. **Tre cloud separati**: Railway (€5/mese), Supabase (gratuito), Netlify (gratuito)
4. **Docker** garantisce che il backend si comporti uguale in locale e in produzione

Stack Cloud

Railway	€5/mese
Supabase	Gratuito
Netlify	Gratuito

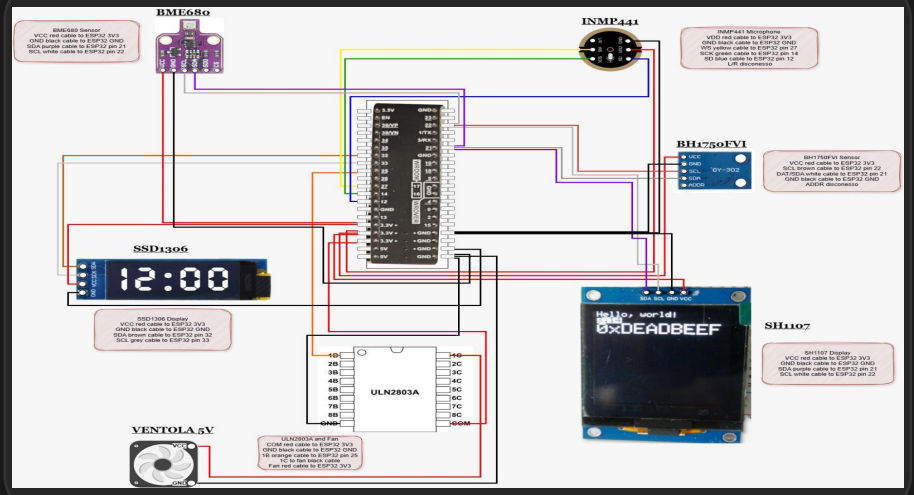
Hardware & Firmware — L'anima fisica del progetto

Componenti Hardware

Componente	Funzione	Interfaccia
ESP32 WROVER 240MHz	Cervello: WiFi, calcolo, trasmissione	core
BME680	Temperatura · Umidità · Pressione · Gas	I²C
BH1750	Luce in lux	I²C
INMP441	Microfono → calcolo dB SPL	I²S
SH1107 128×128 OLED	Schermo 1: tutti i sensori + stato ventola	I²C
SSD1306 128×64 OLED	Schermo 2: orologio NTP in tempo reale	I²C
ULN2803A	Attuatore: controlla la ventola fisica	GPIO 25

Punti Tecnici Chiave

- **I²C protocollo di comunicazione** per i sensori con il microprocessore, usato per tutti i sensori e gli schermi.
- **I²S protocollo di comunicazione** per il microfono INMP441 con il microprocessore, comunica i segnali audio in segnale comprensibili all'ESP32.
- **ULN2803A** circuito integrato servito per la attivazione della ventola evitando sovraccarichi di tensione



Backend — Laravel API in produzione

Endpoint Principali

POST /api/device/data
← ESP32 (no auth — è il device)

GET /api/telemetry
← Dashboard [Bearer token]

PUT /api/device/{n}/fan
← Dashboard [Bearer token]

POST /api/login
← Qualsiasi client

Autenticazione

Laravel Sanctum — token SHA-256

Token stateless : ogni richiesta porta il token, nessuna sessione server-side

Struttura Applicazione

4 Controller

- AuthController
- TelemetryController
- DeviceController

3 Model

- User
- Telemetry
- DeviceSettings

Database & Deploy

PostgreSQL su Supabase — 9 migrazioni evolutive

Docker container → Railway, avvio con Dockerfile

Feature Chiave

Auto-registrazione Device
DeviceSettings::firstOrCreate() — un nuovo ESP32 si "auto-registra" al primo invio senza configurazione manuale

Command-in-Response
La risposta a /device/data contiene fan_desired_state — zero polling extra, zero endpoint separato

Frontend — La Dashboard React

Stack Tecnologico

React 19 · Axios · TailwindCSS 4 · Recharts 3 · Framer Motion · Vite 7

Funzionalità Dashboard

- Login / Registrazione con gestione dei Token
- **6 sensor cards** con valore corrente + unità di misura
- **6 grafici time-series** (ultime 20 letture, Recharts)
- Controllo ventola: toggle **Auto/Manual** + soglia temperatura configurabile
- **Export CSV** dei dati storici
- Deploy: **Netlify** (gratuito)

Dashboard Live

Come comunicano il ESP32 e il server?

ESP32 → POST /api/device/data

Ogni 10 secondi

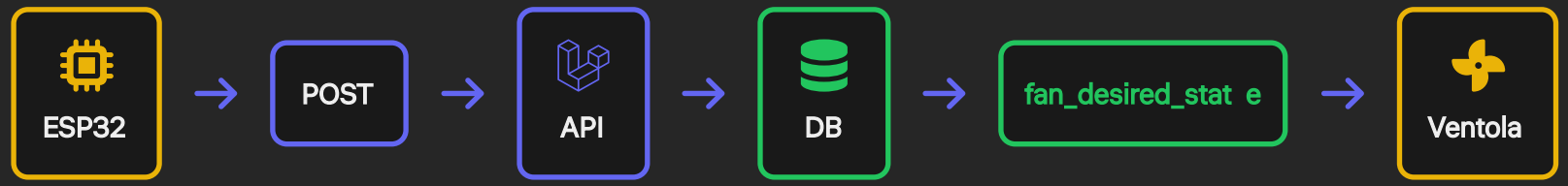
```
{
  "device_name": "esp32_living_room",
  "temperature": 22.5,
  "humidity": 58.3,
  "air_pressure": 1013.2,
  "gas_kohm": 45.7,
  "light_lux": 320.0,
  "noise_db": 48.2,
  "fan_state": false
}
```

Backend → Risposta immediata

Risposta JSON

```
{
  "fan_desired_state": true,
  "message": "Telemetry stored"
}
```

Schema di Comunicazione



Latenza

Backend risponde in **millisecondi** . Latenza totale: rete + PHP ≈ **200-400ms**

Controllo Ventola

Latenza dovuta all'**intervallo ESP32 (10s)** , non al backend

Pattern

"Command-in-Response" — evita endpoint /device/command

Analisi Economica — Quanto costa davvero HomeHero?

Tabella Riassuntiva Costi

Hardware (componenti fisici)	€ 68,50
Cloud e software (2,5 mesi)	€ 12,50
Totale costi materiali	**€ 81,00**

Stima Lavoro Umano

Studenti (~€19/h) Stima realistica	€ 2.830
Professionisti (~€47/h) Stima commerciale	€ 7.425

Break-even Analysis

Prezzo di vendita ipotetico	€ 89/unità
Unità necessarie break-even (anno 1)	~140 unità

Costo variabile per unità:

Hardware: ~€ 68,50 + Shipping: ~€ 8,50 = € 77

Margine per unità	€ 12/unità

HomeHero vs Prodotti commerciali

Confronto Caratteristiche

Prodotto	Prezzo approx	Sensori principali	Controllo attuatore	API aperta
Home Hero (nostro)	prototipo	Temp, Umid, Press, Gas, Lux, dB	Sì	Sì
Xiaomi Mi Air Monitor	~€50	Temp, Umid, PM2.5, CO_2	No	No
Govee Air Quality	~€45	Temp, Umid, tVOC, CO_2	No	Limitata
Airthings Wave Plus	~€270	Temp, Umid, Press, tVOC, Radon	No	Limitata

Messaggio: HomeHero costa meno, fa di più, ed è completamente nostro .

Vincoli, rischi gestiti e stato finale

Vincoli Principali

⏱️ 11 settimane di sviluppo

A tempo parziale, bilanciato con altri impegni

💶 €81 autofinanziati

Interamente da Roman, personalmente

📶 Solo WiFi 2.4GHz

Nessuna connettività 4G/LTE

🔧 No aggiornamenti firmware OTA

Aggiornamenti richiedono connessione fisica

Stato Finale

- Firmware ESP32 ✓
- Laravel API ✓
- PostgreSQL ✓
- React Dashboard ✓
- Deploy cloud ✓
- Documentazione ✓
- Test automatici !

Cosa abbiamo imparato — e cosa viene dopo

Lezioni del Team

R Roman

"Separare backend e frontend fin dal giorno 1 ha evitato ogni problema di integrazione"

L Luka

“Distinguere responsabilita da compassione"

S Shaeek

"Con più dispositivi I²C, i bus separati non sono opzionali — sono necessari"

M Matteo

"Documentare durante lo sviluppo, non dopo, fa la differenza"

D De Togni

"Il GANTT reale diverge sempre da quello pianificato — e questo è normale"

Roadmap Futura

H1 Breve Termine
Prossimi mesi

- Email verification
- Token expiration
- API key ESP32

H2 Medio Termine
6-12 mesi

- Multi-dispositivo
- CI/CD pipeline
- RBAC

H3 Lungo Termine
1+ anno

- App mobile
- MQTT real-time
- Assistente AI

Grazie

HomeHero è live

URL API
hhero.up.railway.app/api

URL Dashboard
hohero.netlify.app

Repository
github.com/xxRelaXxx/HomeHero

Documentazione
ENG · ITA

Domande?